check_paired.py 사용 설명서

1. 개요

src/check_paired.py 는 fasterq-dump --split-files(또는 fastq-dump --split-files)로 내려받은 **paired-end FASTQ 두 파일(_1, _2)이 제대로 짝을 이루는지** 확인하는 스크립트이다.

다음 두 가지를 검사한다.

1. 두 파일의 **read 개수가 같은가?**
2. **같은 순서(n번째)에 있는 read 끼리 이름이 짝을 이루는가?** (끝의 짝 표시 /1, /2 등은 빼고 비교)

paired-end 분석 도구(fastp, shovill, bwa 등)는 _1 의 n번째 read 와 _2 의 n번째 read 가 한 쌍이라고 가정한다. 한쪽 파일만 잘리거나 순서가 어긋나면 결과가 틀어지므로, 분석 전에 이 검사를 해 두면 좋다.

- 표준 라이브러리(sys)만 사용하므로 별도 설치 없이 실행된다.

2. 사용법

lab11 폴더 안에서 실행한다.

```
cd lab11
python3 src/check_paired.py <_1.fastq 경로> <_2.fastq 경로>
```

파일을 두 개 주지 않으면 사용법을 안내하고 종료한다.

2-1. 실행 예시 (정상)

```
python3 src/check_paired.py data/SRR1553607_1.fastq data/SRR1553607_2.fastq
```

출력 결과:

```
파일 1: data/SRR1553607_1.fastq -> read 203445 개
파일 2: data/SRR1553607_2.fastq -> read 203445 개
[통과] 두 파일의 read 개수가 같다.
[통과] 비교한 203445 쌍 모두 이름이 짝을 이룬다.
```

2-2. 실행 예시 (문제가 있을 때)

일부러 틀리게 만든 작은 파일로 시험한 결과이다. 파일 1에는 r1/1, r2/1, r3/1 세 개, 파일 2에는 r1/2, r9/2 두 개의 read 가 있다.

```
파일 1: a.fq -> read 3 개
파일 2: b.fq -> read 2 개
[실패] 두 파일의 read 개수가 다르다. (차이: 1 개)
[실패] 비교한 2 쌍 중 1 쌍의 이름이 짝을 이루지 않는다.
  짝이 안 맞는 예 (순서, 파일1 이름, 파일2 이름):
    2 ¦ r2/1 ¦ r9/2
```

- 짝이 안 맞는 쌍은 **처음 5개까지만** 보여 준다 (코드의 MAX_SHOW 값).
- 한쪽 파일이 먼저 끝나면, 남은 read 는 개수만 세고 이름 비교는 하지 않는다.

3. 이름을 비교하는 방법

FASTQ 헤더 줄에서 다음 순서로 "기본 이름"을 만든 뒤, 두 파일의 기본 이름이 같으면 짝이 맞는 것으로 본다.

1. 맨 앞의 @ 를 뗀다.
2. 공백 앞의 **첫 단어**만 남긴다. (뒤의 설명 부분은 무시)
3. 끝이 **"구분 기호(/ . _) + 1 또는 2"** 이면 그 두 글자를 뗀다.

파일 1 헤더	파일 2 헤더	기본 이름	판정

@SRR1553607.1 1 length=101	@SRR1553607.1 1 length=101	SRR1553607 / SRR1553607	짝 맞음
@read7/1	@read7/2	read7 / read7	짝 맞음
@SRR1553607.5.1	@SRR1553607.5.2	SRR1553607.5 / SRR1553607.5	짝 맞음
@M001:12:FC:1:1101:1000:2000 1:N:0	@M001:12:FC:1:1101:1000:2000 2:N:0	첫 단어가 같음	짝 맞음
@r2/1	@r9/2	r2 / r9	**짝 안 맞음**

- 이번 실습 데이터(SRR1553607)는 --readids 없이 내려받아서 _1, _2 의 헤더가 **완전히 똑같다**. 짝 표시를 떼는 규칙은 다른 형식의 데이터를 위해 넣은 것이다.
- 같은 규칙을 두 이름에 똑같이 적용하므로, 원래 같은 이름은 항상 같은 기본 이름이 된다.

4. 코드 구조

함수	하는 일
read_one_record(f)	파일에서 read 하나(4줄)를 읽고 헤더의 이름을 돌려준다. 파일이 끝났으면 None
base_name(name)	이름에서 첫 단어만 남기고 끝의 짝 표시(/1, .2 등)를 뗀다
main()	두 파일을 read 하나씩 나란히 읽으며 개수를 세고 이름을 비교한 뒤 결과를 출력한다

두 파일을 한꺼번에 메모리에 올리지 않고 **read 하나씩 나란히** 읽기 때문에 큰 FASTQ 파일도 처리할 수 있다.

5. 주의할 점

- FASTQ 가 **정확히 4줄에 read 하나**인 형식이라고 가정한다. 서열이 여러 줄로 나뉜 FASTQ 나 빈 줄이 섞인 파일에서는 엉뚱한 줄을 헤더로 읽는다.
- 압축 파일(.fastq.gz)은 바로 읽지 못한다. 먼저 gunzip 으로 풀어서 사용한다.
- 이름만 비교하고 **서열·품질은 검사하지 않는다.** 서열 길이와 품질 길이가 같은지 같은 검사는 하지 않는다.
- 끝이 1/2 가 아닌 짝 표시(예: _R1, _R2)는 떼지 않는다. 이런 이름이면 짝이 맞아도 "짝 안 맞음" 으로 나올 수 있다.
- 트리밍 등으로 **한쪽 파일에서만 read 를 버린 경우** 순서가 어긋나 그 뒤의 거의 모든 쌍이 "짝 안 맞음" 으로 나온다. paired-end 트리밍은 두 파일을 함께 처리하는 옵션(fastp -i ... -I ...)을 써야 한다.